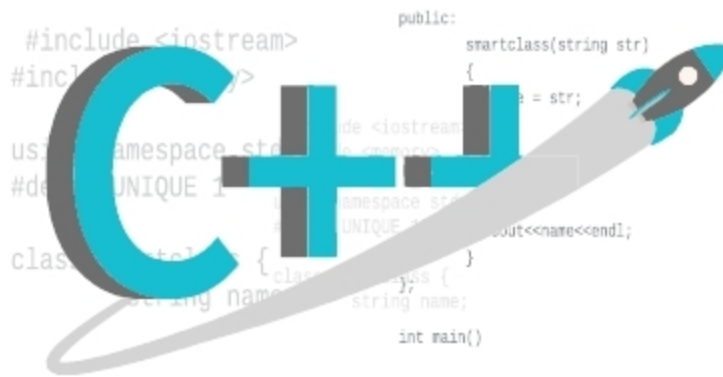


Let's Get Familiar with Smart Pointers in C++

Many readers have probably heard about smart pointers and their benefits. One of the features of C++11, smart pointers are quite useful and have an inbuilt mechanism to ensure a program is free from memory leaks.



Before beginning, let's have a look at why people prefer smart pointers over normal pointers. When dealing with normal pointers, the onus is on programmers to allocate and release the resources or memory, in order to avoid memory leaks and exceptions. Programmers also need to prevent access to the resources that are already freed up, or it will lead to dangling references. That is why many beginners dislike normal or raw pointers.

Smart pointers and their types

Smart pointers are not just pointers; they are objects that are smarter. Unlike normal pointers, memory is released once the pointer goes out of scope.

unique_ptr: An object can be held by a pointer at a given time — multiple pointers cannot point to the same object. For example, consider a pointer *obj1* pointing to an object; another pointer named *obj2* cannot point to the object pointed by *obj1*. *unique_ptr* doesn't allow copying of pointers, but it allows transfer of ownership from one pointer to another.

- Allows exactly single ownership of the object.
- No two different pointers can hold the same object.

- Object cannot be shared or copied to another pointer but can be moved.
- It can manage a single object and dynamically allocate an array of objects.

```
#include <iostream>
#include <memory>

using namespace std;
#define UNIQUE 1

class smartclass {
    string name;

public:
    smartclass(string str)
    {
        name = str;
    }

    int display()
    {
        cout<<name<<endl;
    }
};

int main()
{
    unique_ptr<smartclass> P1(new
    smartclass("smartclass"));
```

```
P1->display();
#ifdef UNIQUE
    unique_ptr<smartclass> P2;
    P2 = move(P1);
#else
    unique_ptr<smartclass> P2(P1);
#endif
P2->display();

return 0;
}
```

The output is:

```
smartclass
smartclass
```

Comment *#define UNIQUE 1* in the above code and compile; then the compiler throws an error. *unique_ptr* doesn't have a copy constructor.

shared_ptr: Of all the smart pointers, this one is really smart. Multiple pointers can point to a single object. This is based on the shared ownership model. For example, pointers *obj1* and *obj2* can point to an object.

- It is based on the shared ownership model.
- It has a reference counting mechanism; when each pointer gets released or goes out of scope the reference count decreases by 1.
- When the reference count reaches 0, the resource is freed up. Reference count can be retrieved using the *use_count()* method. *shared_ptr* manages two blocks:

